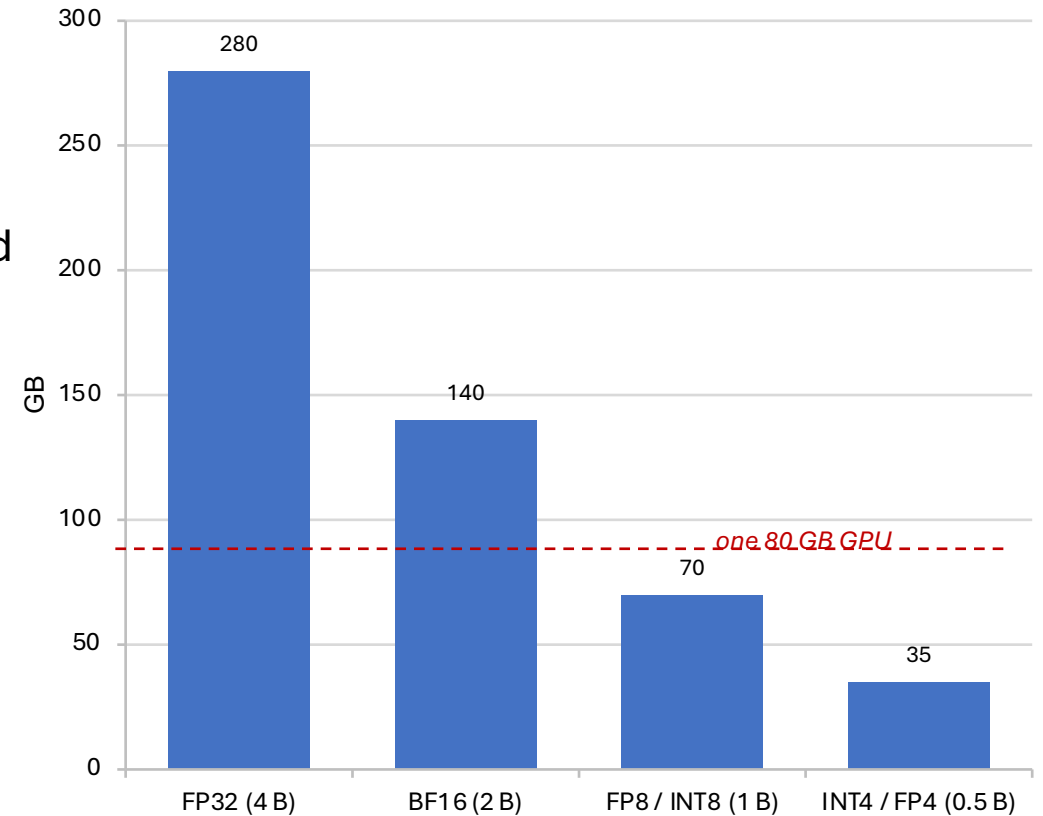


Why efficiency?

- A 70B-parameter model is **140 GB in BF16** (2 bytes/param) — it does not fit one 80 GB GPU
- Decoding is **memory-bandwidth bound**: every generated token re-reads all weights, so $\text{tokens/s} \approx \text{bandwidth} / \text{model bytes}$
- Fine-tuning with Adam in mixed precision needs $\approx$ **16 bytes/param** (fp32 weights + grads + two moments + bf16 copy): ~ 1.1 TB for 70B
- Three levers today: fewer **bits** (quantization), fewer **weights** (pruning, distillation), fewer **trainable weights** (LoRA)
- Slides follow MIT 6.5940 (Song Han, Fall 2024): efficientml.ai



INT4 and FP4

Exponent Width → Range; Fraction Width → Precision

INT4

S			
0	0	0	1
0	1	1	1

=1

=7

-1, -2, -3, -4, -5, -6, -7, -8 0, 1, 2, 3, 4, 5, 6, 7

0 1 2 3 4 5 6 7

-1, -2, -3, -4, -5, -6, -7, -8 0, 1, 2, 3, 4, 5, 6, 7

FP4 (E1M2)

S	E	M	M
0	0	0	1
0	1	1	1

=0.25×2¹⁻⁰=0.5

=(1+0.75)×2¹⁻⁰=3.5

-0, -0.5, -1, -1.5, -2, -2.5, -3, -3.5
0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5

● ● ● ● ● ● ● ●
3 3.5
0 1 2

-0, -1, -2, -3, -4, -5, -6, -7 0, 1, 2, 3, 4, 5, 6, 7×0.5

FP4 (E2M1)

S	E	E	M
0	0	0	1
0	1	1	1

=0.5×2¹⁻¹=0.5

=(1+0.5)×2³⁻¹=1

-0, -0.5, -1, -1.5, -2, -3, -4, -6
0, 0.5, 1, 1.5, 2, 3, 4, 6

no inf, no NaN

● ● ● ● ●
0 1 2 3 4 6

-0, -1, -2, -3, -4, -6, -8, -12 0, 1, 2, 3, 4, 6, 8, 12 ×0.5

FP4 (E3M0)

S	E	E	E
0	0	0	1
0	1	1	1

=(1+0)×2¹⁻³=0.25

=(1+0)×2⁷⁻³=16

-0, -0.25, -0.5, -1, -2, -4, -8, -16
0, 0.25, 0.5, 1, 2, 4, 8, 16

● ● ● ●
0 1 2 4 8 16

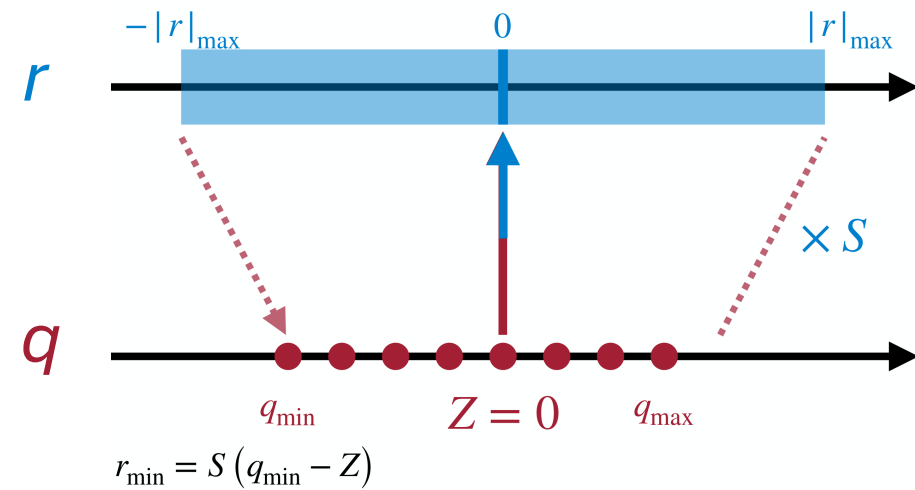
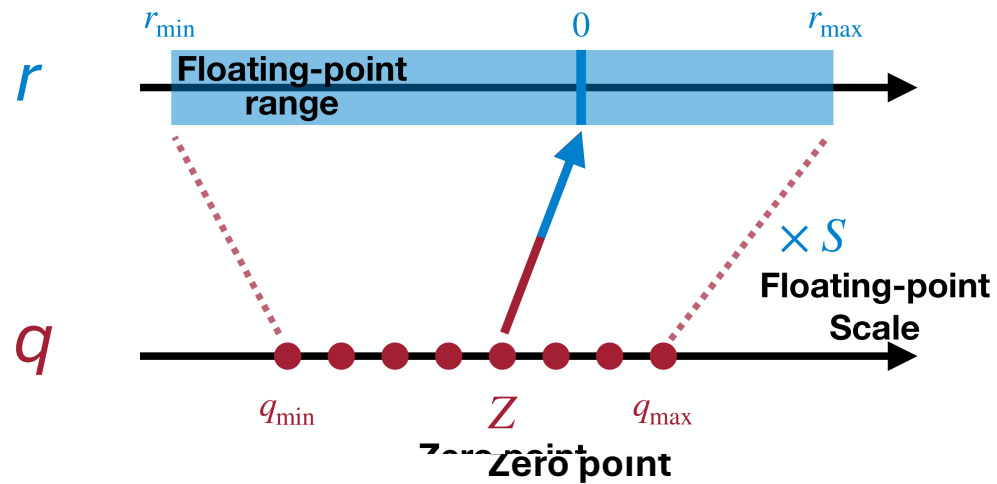
-0, -1, -2, -4, -8, -16, -32, -64 0, 1, 2, 4, 8, 16, 32, 64 ×0.25

Formats in 2024–26 training and serving

- **BF16** is the training default: FP32 range with 8 bits of mantissa
- **FP8 training at scale:** DeepSeek-V3 (Dec 2024) trained a 671B mixture-of-experts model with FP8 mixed precision
- **NVFP4** (FP4 values with per-16-element FP8 scales) — 'Pretraining Large Language Models with NVFP4' (NVIDIA, Sep 2025) on Blackwell GPUs
- **MXFP4 weights:** gpt-oss (Aug 2025) ships its MoE weights in 4-bit so the 120B model fits on one 80 GB GPU

Symmetric Linear Quantization

Full range mode



$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$$S = \frac{r_{\min}}{q_{\min} - Z} = \frac{-|r|_{\max}}{q_{\min}} = \frac{|r|_{\max}}{2^{N-1}}$$

Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7

N-2N-1 2N-1-1

Per-Channel Weight Quantization

Example: 2-bit linear quantization

Per-Channel Quantization Per-Tensor Quantization

OC

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$|r|_{\max} = 2.09 \quad S_0 = 2.09$$

$$|r|_{\max} = 2.12 \quad S_1 = 2.12$$

$$|r|_{\max} = 1.92 \quad S_2 = 1.92$$

$$|r|_{\max} = 1.87 \quad S_3 = 1.87$$

$$|r|_{\max} = 2.12$$

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

$$1 \rightarrow 0 \quad 1 \quad 0 \quad 2.12 \mid 0 \mid 2.12 \mid 0$$

$$0 \rightarrow 0 \quad -1 \quad 1 \quad 0 \mid 0 \mid -2.12 \mid 2.12$$

$$0 \rightarrow 1 \quad 0 \quad 0 \quad 0 \mid 2.12 \mid 0 \quad 0$$

$$1 \rightarrow 0 \quad 1 \quad 1 \quad 2.12 \mid 0 \mid 0 \mid 2.12 \mid 2.12$$

Quantized Reconstructed

$$\|W - Sq_W\|_F = 2.28$$

<https://efficientml.org>

GPTQ and the 4-bit rule of thumb

- **GPTQ** (Frantar et al., 2022): layer-by-layer, second-order (Optimal Brain Surgeon-style) error compensation while rounding; a 175B model to 4 bits in a few GPU-hours
- **Rule of thumb**: 4-bit weights cost little perplexity, 3-bit hurts, 2-bit needs QAT or special codebooks
- 'The case for 4-bit precision' (Dettmers & Zettlemoyer, 2023): across model sizes, 4 bits is Pareto-optimal for zero-shot accuracy vs. total model bits — a bigger model at 4 bits beats a smaller one at 16
- KV-cache quantization (e.g., KIVI, 2-bit, 2024) is the long-context counterpart

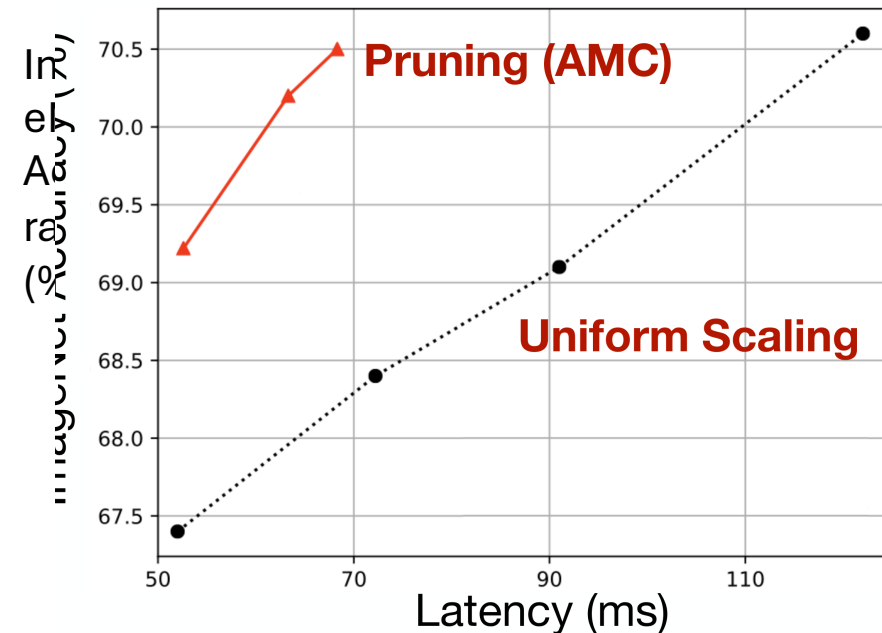
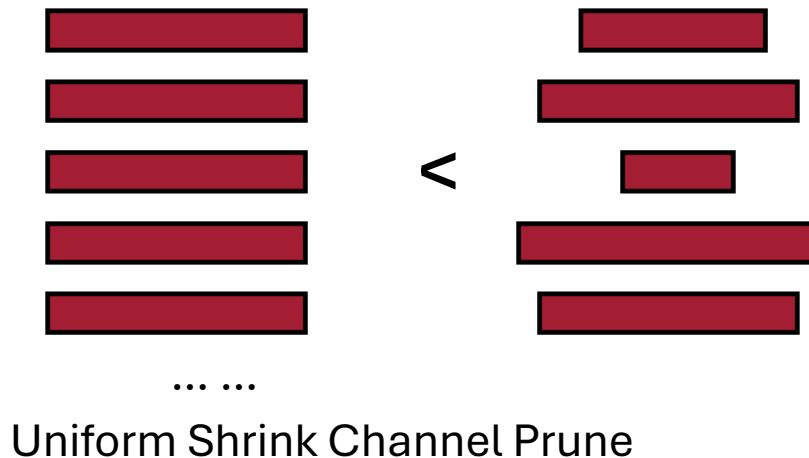
Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**

- Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)

- Con: smaller compression ratio



Magnitude-based Pruning

A heuristic pruning criterion

- ***Magnitude-based pruning considers weights with larger absolute values are more important***

than other weights

- Magnitude

$$\|\mathbf{W}^{(S)}\|_p = \left(\sum_{i \in S} |w_i|^p \right)^{1/p}, \text{ where } \mathbf{W}^{(S)} \text{ is a structural set of parameters}$$

• Example

3 | -2

L2-norm

1 | -5

Row-wise

$\sqrt{13}$
$= \sqrt{ 3 ^2 + -2 ^2}$
$\sqrt{26}$
$= \sqrt{ 1 ^2 + -5 ^2}$

$\sqrt{13}$
$\sqrt{26}$

0 | 0

1 | -5

Weight

Importance

Pruned Weight

Lottery tickets, and one-shot pruning of LLMs

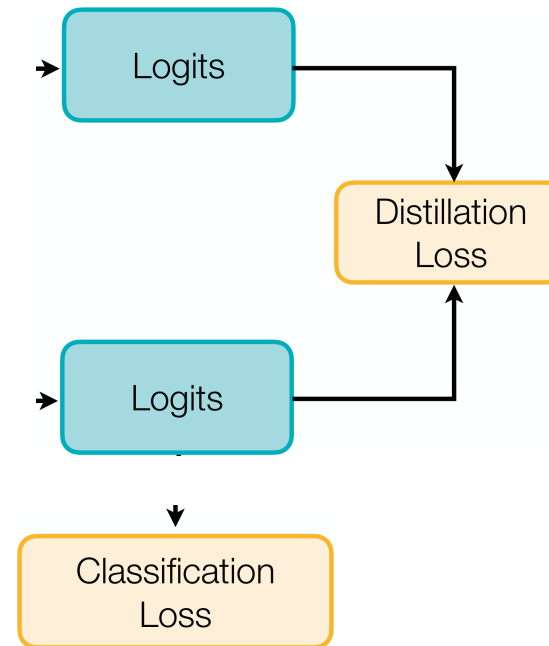
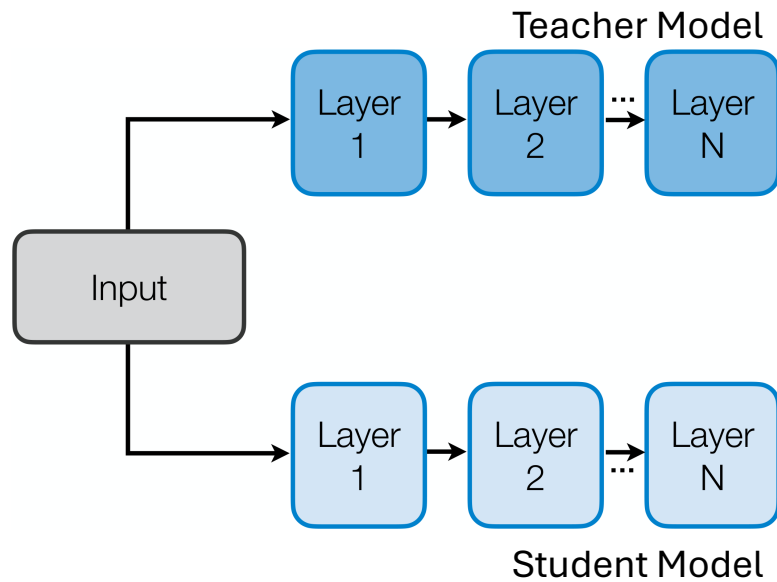
- **Lottery Ticket Hypothesis** (Frankle & Carbin, ICLR 2019 best paper): a dense net contains a sparse subnetwork that, trained alone from the original init, matches full accuracy — found by iterative magnitude pruning with weight rewinding
- **SparseGPT** (Frantar & Alistarh, 2023): 50–60% unstructured sparsity on OPT-175B in under 4.5 GPU-hours, no retraining; **Wanda** (Sun et al., 2023) prunes by $|W| \cdot \|x\|$ with no weight update
- **Structured**: Minitron (NVIDIA, 2024) prunes depth/width of Nemotron-4 15B to 8B/4B and distills with up to 40× fewer tokens than training from scratch; removing up to ~half the layers of Llama-2-70B costs little on QA after light healing (Gromov et al., 2024)

Formal Definition of KD

- **Neural networks typically use a softmax function to generate the logits to class probabilities.**
$$p(z_i, T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Here, $i, j = 0, 1, 2, \dots, C - 1$ classes. T is the temperature, which is normally set to 1.
- **The goal of knowledge distillation is to align the class probability distributions from teacher and student networks.**

Matching output logits



Cross entropy loss:
 $\mathbb{E}(-p_t \log p_s);$

L_2 Loss:
 $E(\|p_t - p_s\|_2^2)$

Distilling the Knowledge in a Neural Network [Hinton et al., NeurIPS Workshops 2014]

Recap: the Challenge of Large KV Cache

The KV cache could be large with long context

- We can calculate the memory required to store the KV cache
- Consider Llama-2-70B (if using MHA), KV cache requires

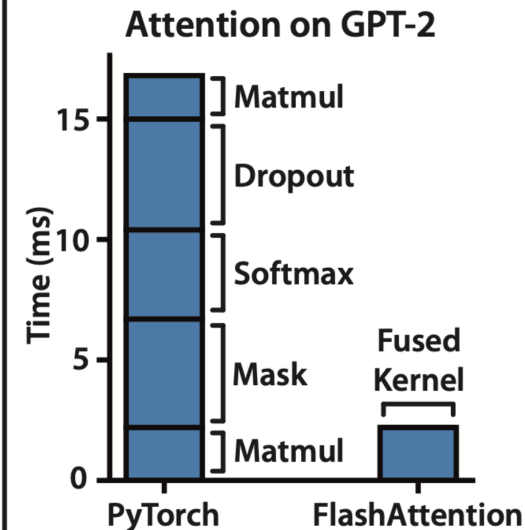
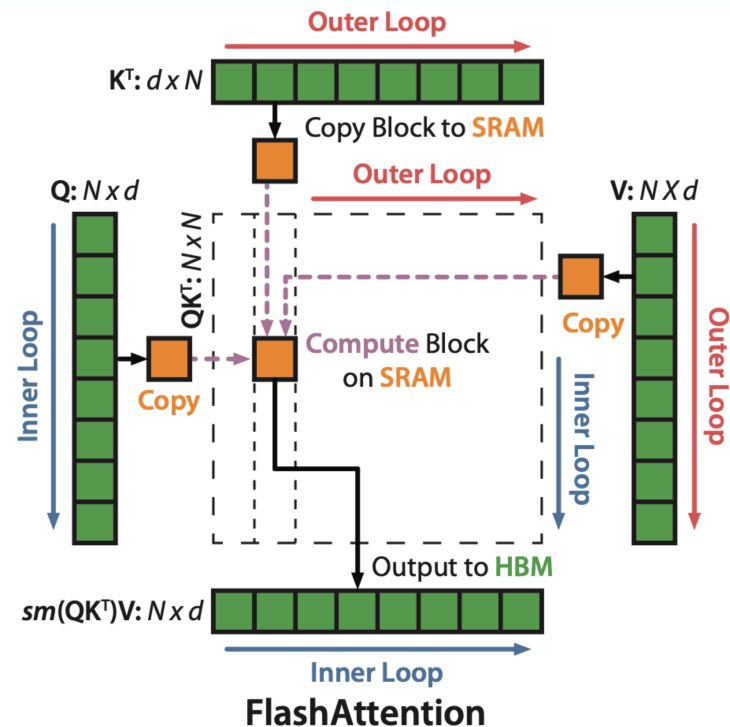
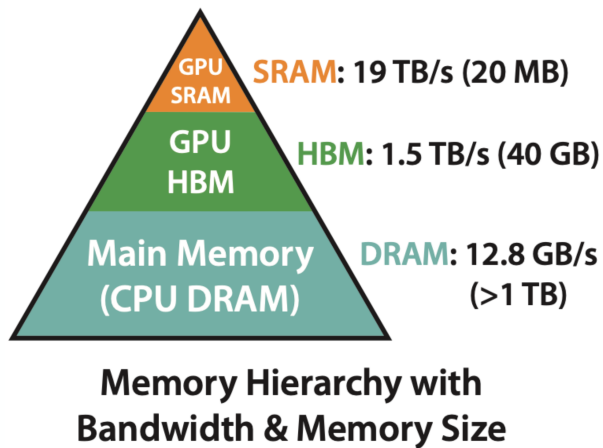
$$\underbrace{BS}_{minibatch} * \underbrace{80}_{layers} * \underbrace{64}_{kv-heads} * \underbrace{128}_{n_{emd}} * \underbrace{N}_{length} * \underbrace{2}_{K\&V} * \underbrace{2bytes}_{FP16} = 2.5MB \times BS \times N$$

- bs=1, n_seq=512: 1.25GB
- bs=1, n_seq=4096: 10GB (~ a paper)
- bs=16, n_seq=4096: 160GB (requires two A100 GPU!)

FlashAttention

- Use tiling to prevent the materialization of the large $N \times N$ attention matrix, thus avoid using slow HBM, kernel fusion.

§



Speculative Decoding

Accelerating memory-bounded generation

- The decoding phase of LLM generates outputs token by token, which is highly memory-bounded
(especially at a small batch size)
- Since multiple tokens are fed to the target model in parallel, it lifts the memory bottleneck
- An example from speculative decoding
- green: accepted, red: rejected, blue: correction

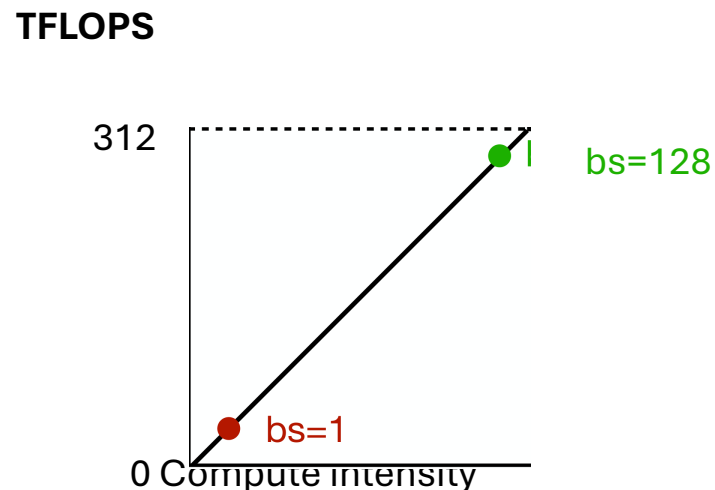
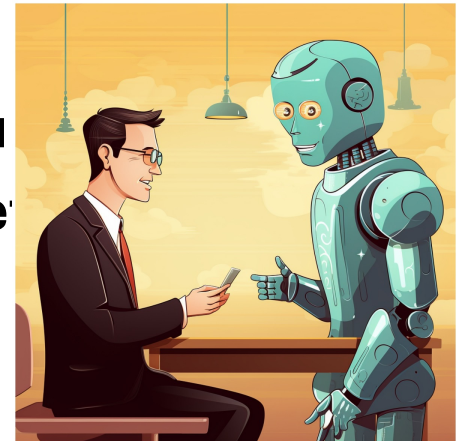
• 2-3x speedup

```
[START] japan ' s benchmark bond n
[START] japan ' s benchmark nikkei 22 5
[START] japan ' s benchmark nikkei 225 index rose 22 6
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 7 points
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 0 1
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 9859
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in tokyo late
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in late morning trading . [END]
```

W4A16 for Single-batch serving

LLM decoding is highly memory-bounded; W8A8 is not enough

- W8A8 quantization is good for batch serving (e.g., batch size 128)
- But single-query LLM inference (e.g., local) is still highly memory-bounded
- **We need low-bit weight-only quantization (e.g., W4A16) for this scenario**



- A100 GPU
- LLaMA-65B decoding